

Model-Based Testing Approach for MATLAB/Simulink using System Entity Structure and Experimental Frames

Artur Schmidt

University of Wismar,
Research Group CEA,
Wismar 23966,
Germany,
artur.schmidt@hs-
wismar.de

Umut Durak

German Aerospace
Center (DLR), Institute
of Flight Systems,
Braunschweig 38108,
Germany
umut.durak@dlr.de

Christoph Rasch

University of Wismar,
Research Group CEA,
Wismar 23966,
Germany,
christoph.rasch@stud.h
s-wismar.de

Thorsten Pawletta

University of Wismar,
Research Group CEA,
Wismar 23966,
Germany,
thorsten.pawletta@hs-
wismar.de

ABSTRACT

In modeling and simulation, Validation and Verification (V&V) have always attracted significant interest. With Model Based Development (MBD) testing models became a part of product V&V. Model-Based Testing (MBT) is an advanced approach for automating the testing process for flexibility and adaptability. MBT advocates utilization of models for the specification of test cases and proposes workflows for automatic test case generation. The paper presents a pragmatic MBT approach for MATLAB/Simulink based on the concept of Experimental Frame (EF) and the System Entity Structure (SES). Each test case is represented following the formal structure of EF. For generating an executable EF, configurable basic models are provided by a Model Base (MB). The SES ontology is then used for the specification of test case designs on an abstract level. It describes a set of various test case structures, parameter settings and objectives. Based on the SES and MB, a specific executable test case, or a test suite, can be automatically generated for a Model Under Test (MUT). Finally, an application, using MATLAB/Simulink, is presented to exemplify the proposed approach.

Author Keywords

model-based testing; verification and validation;
experimental frame; system entity structure;
MATLAB/Simulink.

ACM Classification Keywords

I.6.1 SIMULATION AND MODELING: Simulation
Theory, I.6.4 SIMULATION AND MODELING: Model
Validation and Analysis.

INTRODUCTION

It is more than evident that the advancements in modeling tools and infrastructures have enabled us to construct and execute more complex models. Further, with Model Based Design (MBD), we have located models as the core assets of the development process. But the Verification and

Validation (V&V) of these models still remains an important problem. Testing has long been regarded as one of the most important methods for V&V. Simulating a model under various conditions and comparing its behavior with the system it represents has long been regarded as one of the basic approaches to V&V in the simulation community [6]. Adaptability, flexibility and automation are some of the well-known requirements for software testing infrastructures. Hence, efforts have been made to minimize the huge amount of effort required for testing. Considering the infinite number of scenarios that can be simulated to test a model against the system it represents, these quality attributes are equally well applicable to testing infrastructures for modeling and simulation.

Model Based Testing (MBT) was introduced as a proposal for automating test case generation from a test specification, also called test model, instead of implementing test cases manually [22]. MBT not only automates the testing process, but also enhances the flexibility and adaptability of the testing infrastructure via automating the test case design [19]. Despite the fact that it is widely used in the software testing community, its application in modeling and simulation is quite limited [6].

MATLAB/Simulink is a popular MBD and simulation environment for cyberphysical systems from MathWorks, Inc. [16]. It provides a graphical editor, customizable block libraries and numerical solvers for modeling and simulation of dynamic systems.

The aim of this study is to adopt the MBT practices from software testing to propose a better testing methodology for a widely used simulation environment in industry, namely MATLAB/Simulink, utilizing simulation theory.

Model-based methodologies ask for metamodels to express models. Metamodeling, on the other hand, requires a complete and accurate specification of concepts and a meta-metamodel to express the metamodel. In this study, we adopted simulation theory to fulfill these preconditions. The concept of the Experimental Frame (EF) is employed for formally specifying simulation test cases and System Entity Structure (SES) ontology is used for metamodeling.

The paper first introduces the foundations, including MBT, EF and SES, and then presents the proposed MBT methodology. A representative case study is demonstrated before the conclusion is presented.

BASICS OF MBT AND ITS USAGE IN MATLAB/SIMULINK

MBT often targets the functional testing of a System Under Test (SUT) [20]. SUT is treated as a black box without any knowledge of the internal structure. Only the I/O behavior of the SUT is to be tested.

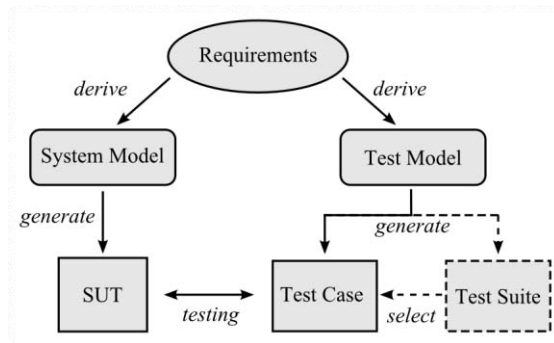


Figure 1. System and test model-driven MBT approach [15].

Figure 1 illustrates the system and test model-driven MBT approach according to [15]. The path on the left side describes the development process of the SUT. Based on the system requirements, a formal system model is derived. It represents a simplification of the structural and behavioral relationships of the components. In the next step, executable components can be generated from the formal system model. These components form the SUT.

The same system requirements are used to derive a formal test model. They describe the intended behavior of the SUT that needs to be tested. From the test model, specific test cases or a collection of test cases, called a test suite, can be generated for a SUT. Briefly, the idea is that test cases are abstracted in a test model, then an MBT tool is employed to generate a set of test cases from that model [19].

In [20], a test case is defined as a sequence of input stimuli to be fed into a SUT, called test inputs, and the expected behavior of a SUT (Figure 2). The expected behavior is often produced using a test oracle. A test oracle contains a judgment unit to make a pass or no pass decision. Various kinds of test oracles can be found in [1].

Early ideas of employing MBT in a MATLAB/Simulink environment were developed by Zander between 2005 and 2008 [21, 9]. Her Model-in-the-Loop for Embedded System Test – Test Harness (MiLEST) infrastructure provides well-structured libraries for test data generation, test control and test validation functions. The SUT and the test specification were both designed in MATLAB/Simulink. Predefined test patterns introduced a systematic test design as a MATLAB/Simulink model on the same layer of the SUT.

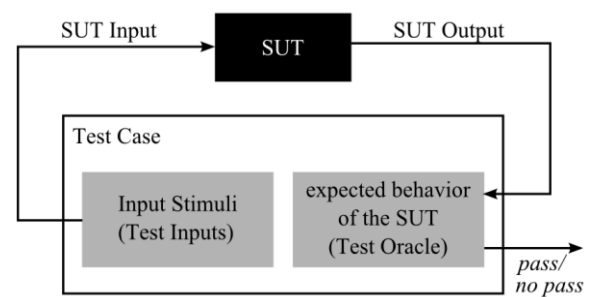


Figure 2. Generic test case structure coupled with a SUT.

On the other hand, since 2006, MathWorks has been providing Simulink Verification and Validation [10] for the realization of MBT in MATLAB/Simulink. As in the MiLEST, Simulink Verification and Validation has also proposed test modeling in a MATLAB/Simulink environment, providing library blocks that target test functions.

While both can be used for testing simulation models for the plants or overall systems, their aim is to provide a methodology to test the controller model that will be used to generate a code to be deployed in an embedded system. Thus, the model that is subject to a test in these two approaches is not necessarily a simulation model. Consequently, supporting the fundamental formalism for simulation models is not the strongest aspect of these two approaches.

On the contrary, in this paper an MBT infrastructure is proposed based on the system theoretical approaches that are adopted in the simulation theory.

MBT USING THE CONCEPTS OF EF AND SES

Zeigler and his colleagues developed system theoretical based approaches for modeling and simulation. In this context, they published in [25, 24] the concept of the Experimental Frame (EF) and the System Entity Structure and Model Base (SES/MB) framework. Subsequently, the concept of EF is employed for the description of test cases and the SES ontology is used for specifying test case structures and test data description. Employing EF and SES in simulation testing was already introduced by Mak, et al. in their Automated Test Case Generator (ATC-Gen) [7]. ATC-Gen is designed to generate test cases to test systems against their conformance to Military Standard 6016C. The minimal Input/Output pairs are transformed into test models and represented in an SES. Then, C++ Discrete Event System (DEVS) code is generated based on SES representation of test models. Mittal, et al. then incorporated and introduced this automated test model generation as part of their DEVS Unified Process [8]. They basically focussed on the testing of net-centric simulations over Service Oriented Architecture.

This paper presents an approach that bridges the gap between the more software engineering oriented approaches

in [10, 21] that target MBD and the system theoretical approaches in [7, 8]. In the following, the concepts of EF and SES are briefly summarized and applied to MBT using MATLAB/Simulink.

Experimental Frame

The concept of experimental frame (EF) was originally introduced by Zeigler [25] in context with the DEVS Specification. The objective was an explicit separation between a dynamic model or system and experiments with it. Moreover, an EF specifies a limited set of circumstances under which a model has to be observed. Nowadays, the EF approach is state of the art and is used in many modelling and simulation projects, including validation investigations [14, 4, 11, 26]. Following Zeigler [25], the formal specification of EF is given by the 7-tupel:

$$EF = \langle T, I, O, C, \Omega_i, \Omega_c, SU \rangle$$

where:

T is the time base,

I is the set of input variables,

O is the set of output variables,

C is the set of control variables,

Ω_i is the set of admissible input segments,

Ω_c is the set of admissible control segments and

SU is a set of summary mappings.

A more formal definition of EF can be found in [18]. The EF can be implemented in various ways.

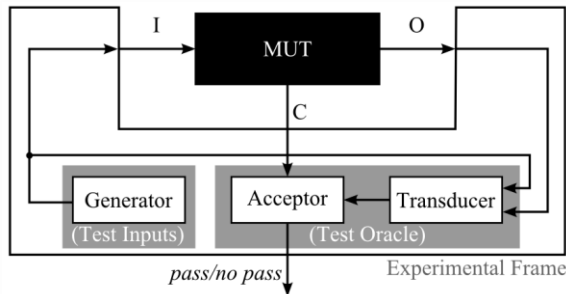


Figure 3. EF with MUT including typical coupling relations [3].

Zeigler [25] recommends the implementation of EF as a coupled model, consisting of a generator, acceptor and a transducer, which are connected to the SUT. In our context, the SUT is always a model. For this reason, we call it Model Under Test (MUT). Figure 3 illustrates such a realization of EF coupled to an MUT schematically. According to the generic test case structure in Figure 2, test inputs are produced by the generator. They have to be admissible input segments of MUT and influence its behaviour. The acceptor and transducer form a test oracle. Based on the output variables, the transducer calculates outcome measures in the form of performance indices, comparative values, statistics etc. The acceptor corresponds

to a decision unit that decides if an experiment is valid or not.

To decide, the acceptor monitors its inputs and maps them to a specified admissible control segment. In the case of a violation of the admissible control segment, the experiment is not accepted. All inputs of an acceptor are control variables.

The EF provides a general structure to define test cases. According to [18], in the case of complex test cases, the components of an EF can be realized in a modular, hierarchical manner and the coupling relations may be more complex.

System Entity Structure and Model Base Framework

The System Entity Structure and Model Base (SES/MB) framework in [24] combines the SES ontology with the classical workflow of modeling and the simulation of modular-hierarchical systems. In addition, it defines the methods for an automatic generation of an executable simulation model. Figure 4 shows the fundamental structure of the SES/MB framework.

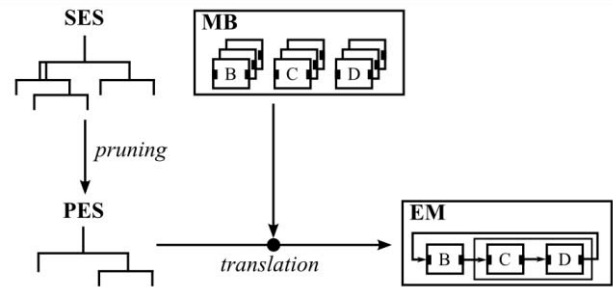


Figure 4. SES/MB Framework based on [24].

Basic models (BM) with a predefined input/output interface are organized in an MB. As stated in [24], BMs describe the dynamic behavior and represent atomic or coupled systems. They are used to compose modular, hierarchical systems. The SES is a high-level ontology, which was originally introduced in [25] for the specification of a set of system structures and parameter settings for modeling and simulation. The SES is represented by a directed and labeled tree with links to BMs in the MB.

Using a *pruning* operation, a distinct system structure can be derived from an SES. The result is called Pruned Entity Structure (PES). Then, a *translation* operation is used to generate an executable simulation model (EM) based on the information of the PES and BMs from the MB.

In the following, SES ontology and pruning as well as translation operation are discussed in more detail and with links to MBT.

System Entity Structure Ontology

The SES provides a limited set of elements and axioms to describe a set of system structures. In [23], four node types,

(i) *entity*, (ii) *aspect*, (iii) *specialization* and (iv) *multiple aspect*, are described for specification of an SES tree. Nodes of type entity represent real or artificial system components [24]. The other node types describe the relationships between their parent and child entities. Aspect nodes have the suffix *Dec* and denote the decomposition relationship of an entity. Specialization nodes represent the taxonomy of an entity (inspired by the *is-a* relationship) and are labeled with the suffix *Spec*. The multiple aspect nodes are labeled with *MAsp* and represent a multiplicity relationship. That means that the parent entity is a composition of multiple entities of the same type.

Figure 5 illustrates an SES specifying eight different test scenarios. The concept test scenario is used here to describe a coupled system containing exactly one MUT and one EF. For pragmatic reasons, the SES in Figure 5 uses some modifications and extensions introduced in [13]. The modifications result from the investigations of various engineering problems in [5, 17].

The node *TestScenarioDec* indicates the decomposition of entity *TestScenario* in the two entities (i) *MUT* and (ii) *EF*. An *EF* consists of a generator (*G*), an acceptor (*A*) and a transducer (*T*). The entity nodes *G*, *A* and *T* are specialized by their successor nodes, *GSpec*, *ASpec* and *TSpec*. *G* is specialized in an entity *Step* or a *RandomNumber* (*RN*), *A* in *InRelativeTolerance* (*IRT*) or *LessThenThreshold* (*LTT*) and *T* in *MeanComputing* (*MC*) or *LastValue* (*LV*).

Attributes can be attached to any node. Aspect nodes, such as *TestScenarioDec* and *EFDec*, define the coupling relationship between their direct predecessors and successors as attributes. The tuple $(MUT.out, EF.in)$ shows that the output of the MUT is connected with the input of the EF, etc. Specialization nodes specify a set of selection rules, labeled with $\{selectrule\}$ and hinted with $\{...\}$, as attributes. An example is depicted for the node *GSpec* in the *Selection Rule* section in Figure 5. They are important for the pruning operation. Leaf nodes specify in their attributes references to BMs in the MB and the parameter setting of BMs. For example, the leaf node *Step* has three attributes, *mb*, *step value* (*sv*) and *step time* (*st*). The attribute *mb* indicates a link to a BM in a corresponding MB.

The remaining attributes of node *Step* represent the parameter setting of the referenced BM of type generator. In this case, the MB contained the following BMs:

$$MB = \{mut, step, rn, irt, ltt, mc, lv\}$$

In the same manner, any test inputs and expected behavior of the MUT or test acceptance criterion can be defined using node attributes.

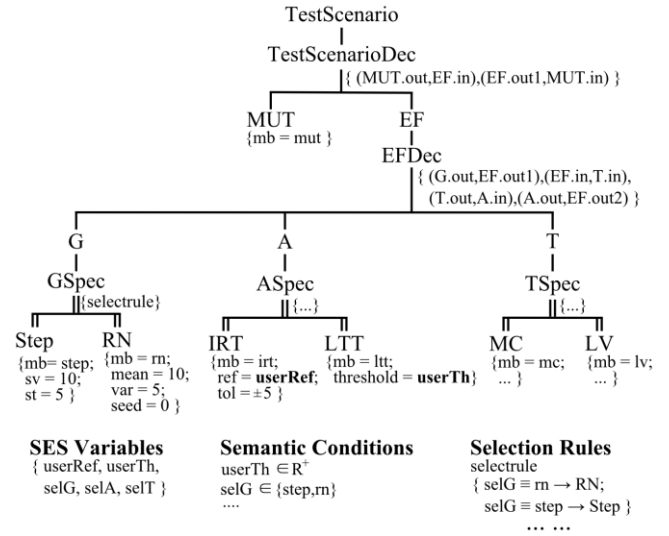


Figure 5. Example of an SES specifying a set of test scenarios.

The pictured *SES Variables* in Figure 5 represent a kind of user interface and are the basis for the pruning operation. They can be separated into two sets: (i) related to the system structure and (ii) related to the parameter setting of the nodes. For example, the listed *SES Variables* $\{selG, selA, selT\}$ belongs to (i) and $\{userRef, userTh\}$ to (ii). Their ranges are limited by *Semantic Conditions*.

In the following, the pruning and translation operations are described.

Pruning and Translation

The SES in Figure 5 describes the variabilities in structure and parameter settings. A *pruning* operation is defined in [24, 23] to derive a distinct and valid system structure with corresponding parameter settings from an SES. The result of the *pruning* operation is a decision-free tree, called *Pruned Entity Structure* (PES).

Before the pruning operation can start, all *SES Variables* must be defined with a distinct value from the range set specified in *Semantic Conditions*. By evaluating the *SES Variables* and *Selection Rules*, all variabilities in structure and parameter setting will be resolved during the pruning operation. For example, the *SES Variable* configuration:

$$SES_Vars = \{selG=step, selA=ltt, selT=mc, userTh=20\}$$

will result in the PES illustrated in Figure 6.

Because of the axiom inheritance, described in [23], the names of the predecessors and successors of specialization nodes are combined. Additionally, the coupling relations are updated in Figure 6.

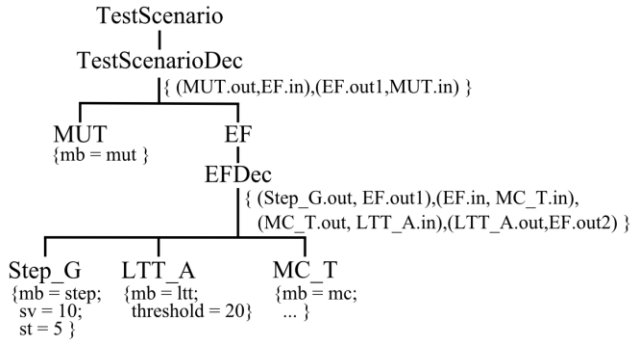


Figure 6. One possible PES of the SES from Figure 5.

The derived PES contains all needed information, such as references to the BMs in the MB, their parameter setting and the modular, hierarchical structure as well as the coupling relationships, to generate an executable test scenario through the *translation* operation. In Figure 7, the modular, hierarchical structure of an executable test scenario is illustrated.

THE MBT INFRASTRUCTURE AND APPLICATION EXAMPLE

Based on the concepts of MBT, EF and the SES/MB framework, a testing infrastructure has been developed and implemented for the MATLAB/Simulink environment. In the following, the infrastructure is employed in an application example.

Infrastructure

Figure 8 illustrates the general structure of the developed test infrastructure. Starting from the requirements specification, an SES needs to be defined for specifying a set of test scenarios and at least one MUT, analogous to Figure 5. The test case structure is based on the general definition in Figure 3. The MB provides a set of BMs, which map different types of generators (G), acceptors (A) and transducers (T). Moreover, the MB contains all the required MUTs for the generation of possible executable test scenarios. These facts are simplified as illustrated in Figure 8.

Source Test Files (STFs) are the user interface for the Test Environment (TE). Using STF, the user needs to specify the MUTs and the test cases that should be performed on the MUTs. A specific MUT_i and EF_j are selected via value assignments to the SES Variables in the STF. In Figure 8, a simplified sketch of the workflow is provided. Referring to the STF box in the workflow sketch, each row of the illustrated STF represents a specific configuration of the SES Variables.

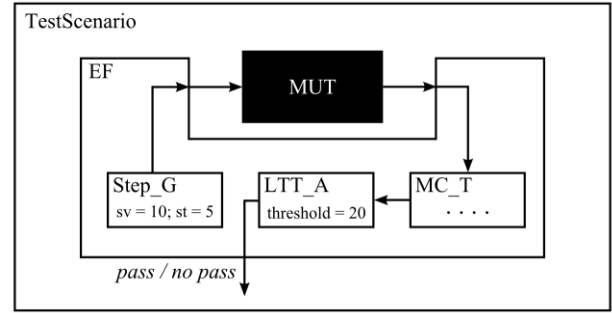


Figure 7. Generated executable test scenario based on the PES and the corresponding MB.

The STF is then sent to the TE which successively selects the desired test from the STF. In the first step, the MUT_i with the corresponding set of EFs is sorted out. The next step selects a single EF_j from this set and a specific SES Variable configuration is picked and sent to the SES/MB framework. Thereupon, an executable test scenario (TS) is generated automatically as a coupled system with the MUT_i and the EF_j , as depicted in Figure 7. The TE then executes the TS, collects the actual test results and proceeds with the next EF_j .

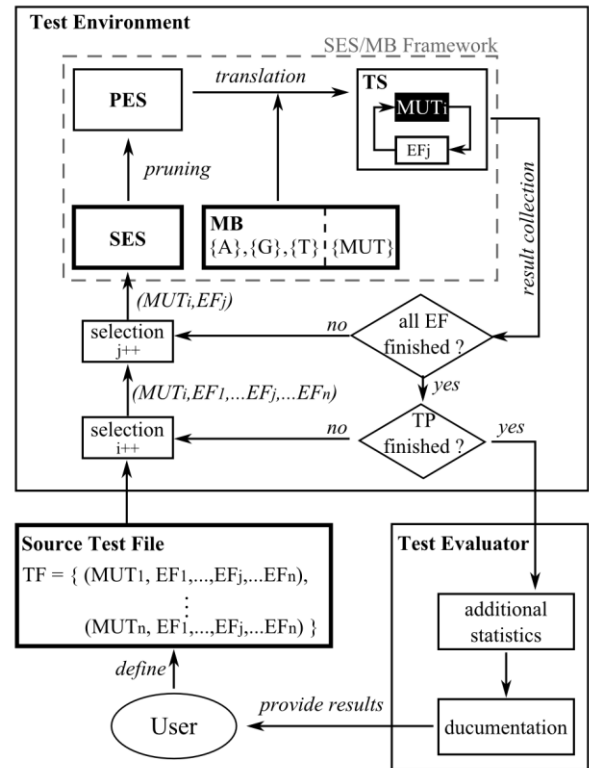


Figure 8. Testing Infrastructure based on the SES/MB Framework.

After all the desired EFs are executed on the MUTi, the next MUTi and EF set are selected from the STF and the test cycle is run again. After finishing the test process, the results of all the tests are transferred to the Test Evaluator. This component computes additional statistics, prepares documentation and provides the results to the user.

The core of the testing infrastructure is the SES Toolbox for MATLAB/Simulink [13]. It comprises a graphical editor for designing system entity structures, pruning and translation methods. The pruning method is based on a modified *Depth-First Search* algorithm for directed graphs. The translation method is built upon the Simulink Application Programming Interface (API) [16].

In the next section, one MUT application is presented to exemplify the workflow.

Application Example

The Research Group Computational Engineering and Automation (CEA) of the Wismar University of Applied Sciences has been developing a Robotic Control & Visualization Toolbox for MATLAB/Simulink, for simulation, control and visualization of six-degrees of freedom robot arms. Currently, the toolbox is usable for robots of type Kawasaki and KUKA. The toolbox consists of two parts: (i) for controlling of simulated and real robot arms [2] and (ii) for simulation of robot arm movements, including handled parts in a virtual environment [12].

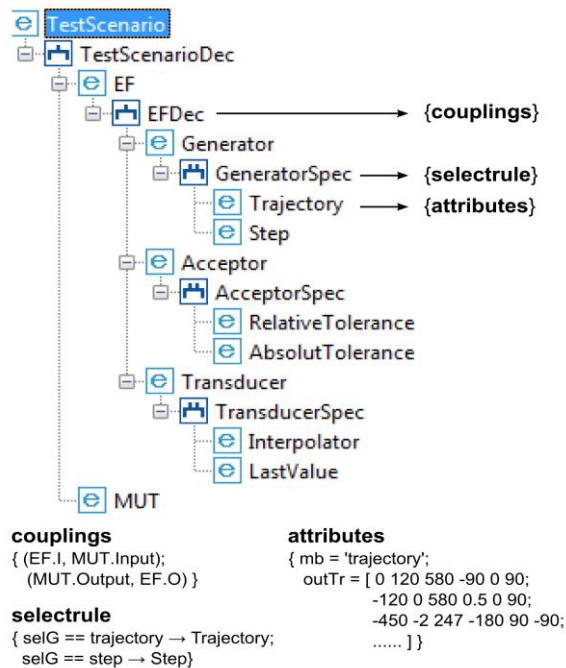


Figure 9. SES tree specifying various test scenarios using the SES Toolbox in [13].

The validation of the simulation for robot arm movements will be presented as an example of the proposed MBT approach. In this validation activity, the Tool Center Point (TCP) positions or trajectories of the real robot will be compared with the simulated one. For this purpose, a test suite is required with numerous test cases.

The simulated robot is treated as the MUT. It computes simulated TCP values (position vector and Euler angles) based on a set of input segments. The required expected results are measured on a real robot using the same input segments. The number of test scenarios composed of exactly one MUT and a set of EFs has been defined using the SES Toolbox for MATLAB/Simulink [13]. Figure 9 illustrates the SES tree.

The entity *Generator* generates test inputs for the MUT and is specialized in two entities: (i) *Trajectory* and (ii) *Step*. *Trajectory* generates a set of desired TCP positions, whereas *Step* generates single TCP positions. For the analysis of MUT outputs, two *Transducers*. (i) *Interpolator* and (ii) *LastValue* are defined. The first continuously refines the inputs for evaluation and the second collects only the last value. Thus, the *Interpolator* is used to analyze the whole trajectory of the MUT and the *LastValue* is used to analyze only the final TCP position of the simulated robot. To make a pass / no pass decision, the *RelativeTolerance* and *AbsolutTolerance* acceptors are defined. The expected results are defined as the node attribute of these acceptors. Both acceptors are designed as alternating acceptors. This means that the acceptors alternate between the pass and no pass states during the test. To generate an executable test scenario, the user has to specify a Source Test File (STF) in MATLAB, defining the desired MUT and the corresponding EF, consisting of one transducer, acceptor and generator. The STF is used to influence the Test Environment and to start the test. Based on the definitions in the STF, a PES is derived from the SES. The subsequent translation operation uses the PES and MB to generate the executable test scenario.

For the validation of TCP trajectory, the *Trajectory* generator, *RelTolerance* with a tolerance of $\pm 5\%$, and the *Interpolator* transducer, are used. The generated MATLAB/Simulink model is shown in Figure 10. The testing results are provided to the MATLAB workspace by *to Workspace* blocks from the Simulink library, which is a part of the *RelativeTolerance_Acceptor* block.

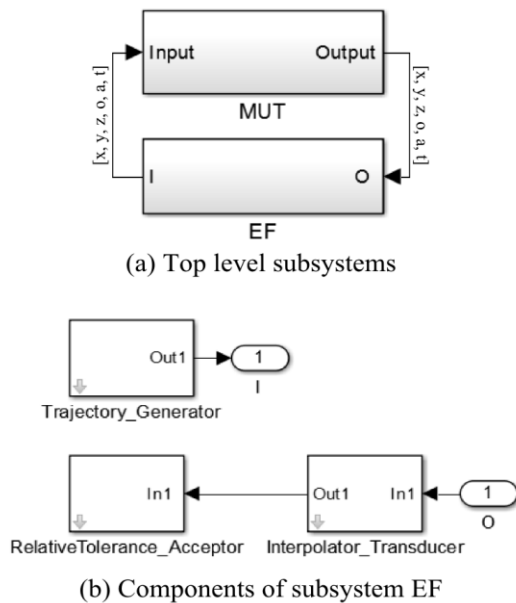


Figure 10. Generated MATLAB/Simulink Model.

Figure 11 illustrates some validation results of one test run. Figure 11a shows the expected versus the simulated TCP trajectory in X direction. Additionally, the inputs, generated by the *Trajectory_Generator*, are marked with a cross. The corresponding pass (1) /fail (0) plot are shown in Figure 11b. The results show that the simulated robot can reach the desired positions. All crosses lie in the state pass. However, the overall trajectory indicates significant deviations from the expected trajectory. Only 32.4% of the overall simulated trajectory lies within the tolerance regarding position.

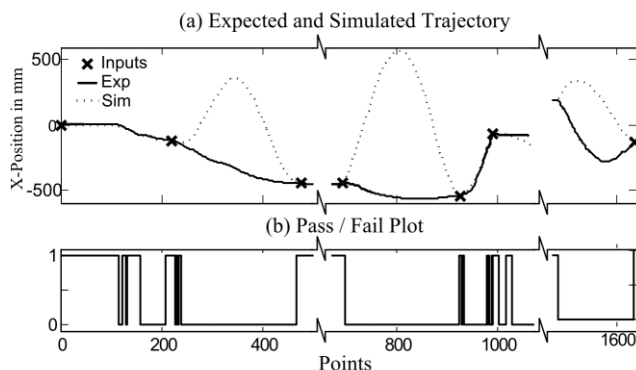


Figure 11. Chosen Test Results.

CONCLUSION

In this study, Model-Based Testing is adopted for effective and efficient testing simulation models in the Model-Based Design environment MATLAB/Simulink. Test models are specified using System Entity Structure and transformed into executable tests employing components from a Model Base that consists of basic blocks for Experimental Frames.

The proposed methodology and developed infrastructure is exercised in a case study in which a robot arm simulation is tested by comparing its results against real robot trajectory. A Model Base that encompasses Experimental Frame components constitutes a reusable asset library for model testing. Therefore, the proposed approach handles adaptability requirements. On the other hand, the transformation tool that automatically generates executable test cases from a test specification model handles efficiency and the effectiveness requirements. The presented case study intends to introduce the practice of the proposed approach, but it will also be useful to collect metrics about reuse ratio and effort reduction.

SES Toolbox is used to model test cases. It is developed as an SES tool for ontology-based simulation in the MATLAB/Simulink. Further customizations of the SES Toolbox may increase efficiency in test case modeling. Likewise, to specify a test scenario, users are proposed to develop Source Test Files. Additional automation will supplement the efficiency of test scenario definition.

Lastly, the proposed infrastructure is intended to be employed on a larger scale to validate flight models in the Air Vehicle Simulator (AVES) research facility of the German Aerospace Center (DLR) Institute of Flight Systems.

ACKNOWLEDGMENTS

This work has been built on a project supported by the German Research Foundation (DFG) under code number PA 631/2-2.

REFERENCES

1. Binder, R. V., (2000). *Testing Object-Oriented Systems: Model, Patterns and Tools*. Addison-Wesley
2. Christern, M., Schmidt, A., Schwatinski, T. and Pawletta, T., KUKA-KAWASAKI-Robotic Toolbox, [Online] Available at: www.mb.hs-wismar.de/cea/KK_Robotic_Tbx/KK_Robotic_Tbx. Accessed on: 10. 10. 2014
3. Durak, U., Schmidt, A. and Pawletta, T., (2014). *Ontology for Objective Flight Simulator Fidelity Evaluation*, In Proc. ASIM 2014, 22. Symposium Simulationstechnik, pp. 93-100
4. Foures, D., Albert, V. and Nketsa, A., (2013), *Simulation validation using the compatibility between Simulation Model and Experimental Frame*, In: Proc. of the 2013 Summer Computer Simulation Conference, Society for Modeling & Simulation International, pp. 55:1-55:7
5. Hagendorf, O., Pawletta, T. and Larek, R., (2013), *An Approach To Simulation Based Parameter And Structure Optimization Of MATLAB/Simulink Models Using Evolutionary Algorithms*, Simulation, 89(9), pp. 1115-1127

6. Hollmann, D., A., Cristia, M. and Frydman, C., (2012). Adapting model-based testing techniques to DEVS models validation. In Proc. of the 2012 Symposium of Theory of Modeling and Simulation – DEVS Integrative M&S Symposium
7. Mak, E., Mittal, S., Hwang, M.H. and Nutaro, J.J., (2010). *Automated Link 16 Testing using DEVS and XML*, Journal of Defense Modeling and Simulation, 7(1), pp. 39-62
8. Mittal, S., Risco-Martin, J., L., Zeigler, B., P., (2009). *DEVS/SOA: A cross-platform framework for net-centric modeling and simulation in DEVS unified process*, Simulation, 85(7), pp. 419-450
9. Model-in-the-Loop for Embedded System Test – Test Harness. [Online_ Available at: www.mathworks.com/matlabcentral/fileexchange/44328-model-in-the-loop-for-embedded-system-test-test-harness. Accessed on : 10.10.2014
10. Model-Based Testing [Online] Available at: www.mathworks.de/discovery/model-based-testing.html Accessed on: 10.10.2014
11. Nader, B. and Filippi, J., B., (2011). *An experimental frame for the simulation of forest fire spread*, In Proc. of the 2011 Winter Simulation Conference, pp.1010-1022
12. Otto, J., Schwatinski, T. and Pawletta, T., (2014). KUKA-KAWASAKI-Visualization Toolbox for MATLAB/ [Online] Available at: www.mb.hs-wismar.de/cea/MatlabKK_Robotic-and-Visualization_Tbx/MatlabKK_Robotic_Visualization_Tbx.html Accessed on: 10. 10. 2014
13. Pawletta, T., Pascheka, D., Schmidt, A. and Pawletta, S., (2013). *Ontology-Assisted System Modeling and Simulation within MATLAB/Simulink*. SNE Simulation Notes Europe, 23(2), p.10
14. Ponnusamy, S., S., Albert, V. and Thebault, P., (2014). *Modelling & Simulation Framework for the Inclusion of Simulation Objectives by Abstraction*, In: Proc. SIMULTECH 2014 – 4th International Conference on Simulation and Modeling Methodologies, Technologies and Applications, pp. 385-394
15. Roßner, T., Brandes, H., Götz, H. and Winter, M., (2010). *Basiswissen Modellbasierter Test*. Heidelberg, dpunkt.verlag
16. Simulink: Getting Started Guide. [Online] Available at: http://www.mathworks.com/help/pdf_doc/simulink/sl_gs.pdf Accessed on: 10.10.2014
17. Schwatinski, T., Pawletta, T. and Pawletta, S., (2012). *Flexible Task Oriented Robot Controls Using the System Entity Structure and Model Base Approach*, SNE Simulation Notes Europe, 22(2), pp. 107-114
18. Traoré, M. and Muzy, A., (2006). *Capturing the dual relationship between simulation models and their context*. In: Simulation Modelling Practice and Theory 14, pp. 126-142
19. Utting, M., and Legeard, B., (2007). *Practical Model-Based Testing: A Tools Approach*. Morgan Kaufmann Publishers Inc .
20. Weissleder, S., (2010). *Test Models and Coverage Criteria for Automatic Model-Based Testing Generation with UML State Machines*, Diss., Humboldt University zu Berlin
21. Zander, J., (2008). *Model-based Testing of Real-Time Embedded Systems in the Automotive Domain*, Diss., TU Berlin
22. Zander, J., Schieferdecker, I. and Mosterman, P., (2012). *Model-Based Testing for Embedded Systems*. CRC Press Taylor & Francis Group
23. Zeigler, B., P. and Hammonds, P., E., (2007). *Modeling and Simulation-Based Data Engineering: Introducing Pragmatics into Ontologies for Net-Centric Information Exchange*. Academic Press
24. Zeigler, B. P., Praehofer, H. and Kim, T. G., (2000). *Theory of Modeling and Simulation: Integrating Discrete Event and Continuous Complex Dynamic Systems*, 2nd ed. Academic Press
25. Zeigler, B. P., (1984). *Multifaceted Modelling and Discrete Event Simulation*. Academic Press Professional Inc.
26. Zengin, A. and Ozturk, M., (2012). *Formal verification and validation with DEVS-Suite: OSPF Case study*, In Simulation Modelling Practice, vol 29, pp 193-206